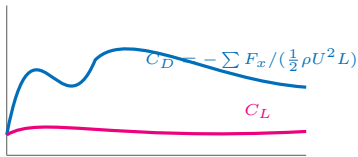
OUTPUT AND ANALYSIS OF FORCES

What IBAMR's forces are, how the examples write them and the positions to disk, and how to read, sum, normalize and filter them in Python and MATLAB



hier_data_IB2d/F.NNNNN

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 11

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say what the numbers in IBAMR's Lagrangian force are (forces, not force densities; the force the structure exerts on the fluid) and what the sum over a structure means (the tether and applied forces; minus the force of the fluid on the body);
2. find and read the `X.NNNNN` and `F.NNNNN` files that the examples write (format, ordering, time stamps), in Python and in MATLAB, with the readers shipped in `examples/analysis`;
3. turn the summed force into a drag or lift coefficient, with the course's $\frac{1}{2}\rho U^2 S$ and chapter 8b's $\rho U^2 L$, and know where the factor 2 and the minus sign come from;
4. measure kinematics from the positions (a tip, a marker cloud, a structure's extent), as every figure of chapters 7–10 did, and filter a noisy force signal without hiding the physics;
5. know the other outputs IBAMR offers (the Silo Lagrangian data for VisIt, the upstream examples' own drag and lift, callbacks) and where chapter 33 takes the Eulerian fields into MATLAB.

> The forces are the quantity a biologist or an engineer wants from an IB simulation: thrust, drag, lift, work. Everything in this chapter is a few lines of reading and summing; the care goes into signs, factors and ordering.

HOW THIS CHAPTER WAS CHECKED

- The file format is taken from files written by the examples with IBAMR 0.19.0 on 1 and 3 processes (slide 8), and the writer is the examples' own `output_data()`, quoted in full; the course's slides and scripts assumed a layout ("three header lines") that holds for the files they had and is checked here against the current ones.
- The readers (`hier_data.py`, `read_hier_data.m`) and the analysis of the tethered leaf (`leaf_forces.py`, `leaf_forces.m`) were run on IB/03's output with Python 3.12 and Octave 8.4; the table and figure are theirs, and the numbers agree between the two.
- The 2013 claim "there is a scaling error in IBAMR, so all forces need to be divided by 2" is examined on slide 15: the factor is the $\frac{1}{2}$ of the dynamic pressure in the coefficient's definition, and the course's own script applied it as such; no later edition repeated the claim.
- The course's `Example_FruitFly_Re10` (`Example_output_forces.zip`) is no longer in the folders; its figure and its script (`dataread2.m`, from the public `Reading-Forces` folder) are described and quoted, and the method is demonstrated on IB/03 instead.

> Slide numbers refer to this deck; chapter numbers (6, 7, 8b, 9a, 9b, 10, 16, 26, 33, 36) to the other chapters of this series.

WHERE TO LOOK THINGS UP

- The writer: `output_data()` at the end of every example's `main.cpp` of this series (slide 10); it comes from IBAMR's `examples/IB/explicit/ex1`, which also shows how to write the Eulerian fields to HDF5.
- The readers: `examples/analysis/hier_data.py` (Python: `read_vec`, `read_series`, `read_input`) and `read_hier_data.m`, `read_input.m` (MATLAB/Octave), with `leaf_forces.py` and `leaf_forces.m` as worked examples; the course's `dataread2.m` and the lab's 3D script are quoted on slides 16 and 25.
- The input keys: `data_dump_interval` and `data_dump_dirname` in the `Main` block of every input file; `viz_dump_interval` next to them (chapter 7's input tour).
- What the force is: the IBAMR FAQ, https://github.com/IBAMR/IBAMR/wiki/FAQ ("forces, not force densities"); chapter 6 (the Δs in every stiffness), chapter 8b (the dimensionless forces).
- Eulerian data into MATLAB: chapter 33 (the `importsamrai` tools of the 2013 lecture, VisIt's export); chapter 26 for VisIt itself.
- The class documentation, https://ibamr.github.io/docs: `LDataManager`, `LData`, `LSiloDataWriter`.

FORCES, NOT FORCE DENSITIES

From the IBAMR FAQ, quoted in every edition of this lecture (2012, 2013, 2015, 2017, 2020):

> *"Although force densities are more natural in the continuum setting, the Lagrangian data structures implemented in IBAMR are not sophisticated enough to use force densities. To simplify the software interface, we decided to make all Lagrangian forces actual forces (not force densities). Note that this convention affects the units required for spring constants, bending stiffnesses, etc.*
>
> *Essentially, if you have a Lagrangian elasticity model that is phrased in terms of force densities, you must convert it into a model that is phrased in terms of forces in order to use this model with IBAMR. If* $\mathbf{F}$ *is the force passed to IBAMR and* $\tilde{\mathbf{F}}$ *is the original force density, what you need to compute is something like* $\mathbf{F}(s,t) = \tilde{\mathbf{F}}(s,t)\,dV$*, where* dV *is the appropriate (discrete) volume element."*

"The new force version of IBAMR will output the forces applied to the fluid. Note these are also forces (not force densities)." (2012) Every IBAMR since has had this; "the new force version" was the 2012 build at UNC.

WHAT THAT MEANS FOR YOUR MODEL AND YOUR FILES

- Every number in `F.NNNNN` is a force on one point (N in SI; per unit depth in 2D, so N/m), not a force per unit length or area. Summing the numbers of a structure gives a force; integrating is not needed, since dV is already inside.
- The Δs factors of chapter 6 are exactly the "$\tilde{\mathbf{F}}\,dV$" conversion: a spring stiffness K per unit strain becomes $K/\Delta s$ in the file, a bending stiffness k_b becomes $k_b/\Delta s^3$, a tether stiffness κ becomes $\kappa/\Delta s$ (IB/03's generator comments say which). Get the factor wrong and the file's forces are wrong by that factor, with no warning.
- The force in the file is the force the structure exerts on the fluid: it is what IBAMR spreads onto the grid. The force the fluid exerts on the structure is its negative (slide 12).
- Chapter 8b's dimensionless force, $F' = F/(\rho U^2 L)$ in 2D or $F/(\rho U^2 L^2)$ in 3D, divides this summed force; the course's coefficient (slide 15) differs by the $\frac{1}{2}$.

> A 2D IB simulation is a slice of unit depth: its "force" is a force per unit depth, and a 2D drag coefficient uses $S = L$, the body's length in the plane. To compare with a 3D measurement you need the depth, or you compare coefficients.

IBAMR FORCE OUTPUT: THE HIER_DATA FOLDER

"Through most of your simulations, you may have noticed a folder called `hier_data_IB2d`. This folder contains:

- Force on each boundary point: `F.01000` for 1000th timestep.
- Position of each boundary point: `X.01000` for 1000th timestep.

The force data is organized so that each component of the force on each boundary point is given in a list. This list is written every N time steps, as specified in the input file. The forces are ordered in the order of the Lagrangian indices. In 2D, the x-components of the force are the odd entries and the y-components of force are the even entries. The position data is similarly organized." (2012–2020)

What the course did not say, because in 2012 it was true by construction:

- these files are not a stock IBAMR output. They are written by `output_data()` in the example's own `main.cpp`, which the course copied from IBAMR's `examples/IB/explicit/ex1` (slide 10). An IBAMR program without that function writes no `hier_data` folder, and a program with a different one writes something else;
- "odd and even" are MATLAB's 1-based positions: in 0-based Python, x is `[0::2]` and y `[1::2]`; in 3D it is three numbers per point;
- the file number is the time step, with leading zeros to five digits (`%05d`): `X.00800` at step 800, `X.10000` at step 10,000, and the time is the step times `DT`;
- "in the order of the Lagrangian indices" means all structures together, in the order of `structure_names` (chapter 9a's index ranges).

THE FILE FORMAT, FROM A FILE

```
Vec Object: 3 MPI processes
  type: mpi
Process [0]
-0.0500028
1.071e-05
-0.0490208
...
Process [1]
...
Process [2]
...
```

`X.00800` of IB/09 (103 points, 3 processes, 0.19.0): two header lines, then each process's block, introduced by `Process [k]`, with one number per line, x then y of each point. The point order is Lagrangian (the `scatterPETScToLagrangian` in the writer), not the order the processes hold them in; the `Process` lines are where the file was assembled from.

These are PETSc ASCII vectors (`VecView` with an ASCII viewer), and their header has changed over PETSc versions: the course's `dataread2.m` skipped three header lines and every line starting with `P`, which fits this layout and the 2013 one ("`Vector Object:`", "`type: mpi`", "`Process [0]`"). A file written by one process has "`1 MPI process`" and `type: seq`, and PETSc may omit the `Process` line: three fixed header lines would then eat the first number.

The readers of this series therefore keep *every line that is a number* and skip the rest, whatever the header (slides 18, 19); a count check (a multiple of `NDIM`) catches a damaged file.

The same holds for `F.NNNNN`; the two files of one step have the same length and order, so the force on point i sits at the same lines as its position.

SETTING HOW OFTEN THE FILES ARE WRITTEN

“Open the `input2d` file and go to the following line:

```
// hierarchy data dump parameters
   data_dump_interval = 1000          // set to
  zero to turn off
   data_dump_dirname = "hier_data_IB2d"
```

`data_dump_interval` sets the number of time steps between force outputs. It is usually worthwhile to make this more frequent than visit outputs for filtering.” (2012–2020)

In the examples of this series (the `Main` block of `input2d`):

```
   viz_dump_interval           = VIZ_INTERVAL     // int(
END_TIME/(50*DT)): 50 frames
   viz_dump_dirname            = "viz_IB2d"
   data_dump_interval          = DATA_INTERVAL    // the same,
50 files
   data_dump_dirname           = "hier_data_IB2d"
```

- the program reads these keys itself (`app_initializer->getDataDumpInterval()`) and calls `output_data()` every that many steps; 0 turns it off;
- 50 files per run is enough for the figures of chapters 7–10; a force signal that oscillates (the fruit fly’s wingbeat, the pump’s pulse) wants 10–20 per period, and a filter (slide 24) wants more;
- each file costs a few lines per point: 860 points and 50 files are 2 MB; 3D structures and 1000 files are not.

THE WRITER: OUTPUT_DATA() IN EVERY EXAMPLE

```
// main.cpp of the examples (from IBAMR examples/IB/explicit/ex1): called every data_dump_interval steps
void output_data(Pointer<PatchHierarchy<NDIM> > patch_hierarchy, Pointer<INSHierarchyIntegrator>,
                 LDataManager* l_data_manager, const int iteration_num, const double loop_time,
                 const string& data_dump_dirname)
{
    const int finest_hier_level = patch_hierarchy->getFinestLevelNumber();
    PetscViewer viewer;
    Pointer<LData> X_data = l_data_manager->getLData("X", finest_hier_level);
    Vec X_petsc_vec = X_data->getVec();
    Vec X_lag_vec;
    VecDuplicate(X_petsc_vec, &X_lag_vec);
    l_data_manager->scatterPETScToLagrangian(X_petsc_vec, X_lag_vec, finest_hier_level);  // Lagrangian order
    file_name = data_dump_dirname + "/" + "X.";
    std::snprintf(temp_buf, sizeof(temp_buf), "%05d", iteration_num);                    // X.00800
    file_name += temp_buf;
    PetscViewerASCIIOpen(PETSC_COMM_WORLD, file_name.c_str(), &viewer);
    VecView(X_lag_vec, viewer);                                                           // the ASCII file
    PetscViewerDestroy(&viewer);
    VecDestroy(&X_lag_vec);
    // ... the same for F_data = l_data_manager->getLData("F", ...) into F.NNNNN ...
    // Sum up the forces: springs and beams pull their points with equal and opposite
    // forces, so only the target-point (and applied) forces add up.
    double F_sum[NDIM]; ... VecGetArray(F_petsc_vec, &F_array); ... F_sum[d] += F_array[NDIM * k + d]; ...
    IBTK_MPI::sumReduction(&F_sum[0], NDIM);
    pout << "integral{F} ="; for (d...) pout << " " << F_sum[d]; pout << "\n";
}
```

The full function is at the end of every example's `main.cpp`; the positions part is shown, the forces part is the same with `"F"`, and the sum follows. Read it on the next slide.

THE WRITER, READ

- `"X"` and `"F"` are the names under which the `LDataManager` keeps the positions and the Lagrangian force of the finest level (the structures all live there);
- `scatterPETScToLagrangian` reorders the vector from the parallel layout to Lagrangian index order before writing, which is why the files are in point order whatever the number of processes;
- `VecView` to an ASCII viewer gives the format of slide 8; a binary viewer would give smaller, faster files that MATLAB and Python can also read (`PetscBinaryIO`), the course never needed it;
- the sum printed as `integral{F}` is the same sum the readers compute (slide 12): a check that a reader is right is to reproduce the log's numbers.

The upstream `ex1` also writes the Eulerian velocity and pressure to HDF5 here; the series' examples dropped that and leave the fields to VisIt (chapter 26) and chapter 33. To write the velocities of the points as well, duplicate the `"X"` block with `getLData("U", ...)` into `U.NNNNN`.

WHAT THE SUM OF THE FORCES MEANS

The Lagrangian force on point i is the sum of what its springs, beams, tethers and any force function of chapter 9b give it. Over a whole structure:

- a spring pulls its two points with equal and opposite forces: $+\mathbf{F}$ and $-\mathbf{F}$, sum zero;
- a beam gives $+2\mathbf{F}$ to its middle point and $-\mathbf{F}$ to each neighbour (chapter 10): sum zero;
- a target point gives $\kappa(\mathbf{X}^{\text{target}} - \mathbf{X})$ to one point and nothing to any other: it does *not* cancel;
- an applied force of chapter 9b between two points of the structure cancels; a uniform body force does not.

So $\sum_i \mathbf{F}_i$ over a structure is the tether force (plus any body force): the force that holds or moves the structure against the fluid. By Newton's third law it is minus the force the fluid exerts on the structure, since the structure is massless (chapter 13 adds mass) and the internal forces cancel. For a tethered body in a stream, $-\sum_i F_{i,x}$ is the drag in the stream's direction and $-\sum_i F_{i,y}$ the lift; for a free swimmer (IB/10) the sum is zero to round-off, which the log shows as `integral{F} = 1e-14`.

> The chapters before this one read the sum off the screen: chapter 7 (-4.7 N on the leaf), chapter 9a (the plates' $63{,}000$ N), chapter 9b (the pump's tethers). This chapter reads it from the files, per structure and per point.

THE COURSE'S EXAMPLE: A FRUIT-FLY WING AT $\mathrm{Re} = 10$

"`Example_FruitFly_Re10`. This example includes files to simulate a flapping plate that uses a motion similar to a fruitfly. Download the files from the Examples folder as `Example_output_forces.zip`. You need to include the following flag when submitting your job to ensure convergence in time:

```
mpirun ./main2d input2d -stokes_ksp_rtol 1.0e-12
```

`Dataread2.m` reads the force files, normalizes the data in terms of force coefficients, and makes a plot of the normalized forces over time. An important component of this code is that it skips over lines that provide processor information." (2012–2020)

- The zip is gone from the folders; the plate's motion was chapter 9a's moving target points with a fruit-fly stroke (translation and rotation at $\mathrm{Re} = 10$), and its figure (next slide) is the model for every force plot of this series.
- `-stokes_ksp_rtol 1.0e-12` tightens the relative tolerance of the Stokes (fluid) solve from its default. Forces are sensitive to the pressure, and the pressure to how well the saddle-point system was solved; a loose tolerance shows up as noise in the force, not in the pictures. Chapter 16 has the other PETSc options; the examples of this series use the default and filter (slide 24).
- "Skips over lines that provide processor information": the `Process [k]` lines of slide 8.

THE COURSE'S FIGURE, DESCRIBED

"`Dataread2.m`. This Matlab code reads the force data and will produce the following graph. Note that the force data is filtered, and this becomes more important at higher Re." (2012–2020)

The graph (not reproduced here, since the example that made it is gone and its title carried a personal path): *Non-Dimensional Force* against *Time (wingbeats)*, from wingbeat 2 to 4, with dashed lines at every half wingbeat. Two raw curves, cyan (horizontal) and magenta (vertical), with the filtered curves in black over them:

- the horizontal force swings between about $+6$ and -6 once per half stroke, as the plate reverses: the dominant force on a flapping wing at $\mathrm{Re} = 10$ is the drag of the stroke, which changes sign with the direction of travel;
- the vertical force stays near $+1$ to $+1.5$ through each half stroke with a dip to -1 at the reversals: the lift, small and positive on average;
- the raw curves show a fast ripple of a few tenths that the 9-point moving average removes; it is largest just after each reversal.

In the script's normalization (next slide) a value of 6 means a horizontal force six times $\frac{1}{2}\rho U^2 L$ with $U = 0.5$ m/s and $L = 0.1$ m: drag coefficients of order 5 are what a plate normal to a stream at $\mathrm{Re} \sim 10$ gives (the cylinder's C_D at $\mathrm{Re} = 10$ is about 3).

CALCULATION OF FORCE COEFFICIENTS, AND THE “DIVIDE BY 2” OF 2013

“Since IBAMR uses forces rather than force densities, summing the horizontal and vertical forces over the wing will give the total horizontal and vertical force acting on the wing, respectively. To calculate the lift and drag coefficients, one would apply the following formula for high Re:

$$F_{L,D} = \tfrac{1}{2}\, C_{L,D}\, \rho\, S\, U^2.$$

Note that in 2D, $S = L$ (the length of the plate).” (2012–2020)

The 2013 edition added: “There is a scaling error in IBAMR, however, so that all forces need to be divided by 2!” No later edition kept the sentence, and the course’s own script never divided by 2: `dataread2.m` computes `-2*sum(A)/(density*velocity^2*length)`, i.e. $C = -\sum F/(\tfrac{1}{2}\rho U^2 L)$. The 2 is the $\tfrac{1}{2}$ of the dynamic pressure, and the minus turns the force on the fluid into the force on the wing.

> Two conventions, both in use in this series: the course’s coefficient $C = F/(\tfrac{1}{2}\rho U^2 S)$ (the aerodynamic one) and chapter 8b’s $F' = F/(\rho U^2 L)$, exactly half of it. Say which one a plot uses. The 2013 “$F = C\rho S U^2$” of the MATLAB lecture (without the $\tfrac{1}{2}$) is the second convention, not a third.

THE COURSE'S READER: DATAREAD2.M

```
path_name=pwd;          %Path name to the folder of interest
starting_time=200000;  %First time step to be included in data analysis
time_step=1000;         %Interval between outputted time steps (this should match the interval in the input2d file)
final_time=4000000;     %Final time step to be included in data analysis
timestep=10^-6;
density=1000; velocity=0.5; length=0.1;   %fluid density; characteristic velocity (m/s); characteristic length (m)
n=1;
for k=starting_time:time_step:final_time
    fileID = fopen([path_name '/hier_data_IB2d/F.' num2str(k)]);
    C=textscan(fileID,'%f','HeaderLines',3,'delimiter',' ','commentstyle','P'); %Scans the file for numbers (skipping 'Processor'
  lines)
    A=C{1};
    fclose(fileID);
    %note that force is the "non-dimensional" force, or force coefficient.
    force(n,1)=-2*sum(A(1:2:end))/(density*velocity^2*length);  %horizontal force (odd numbered entries)
    force(n,2)=-2*sum(A(2:2:end))/(density*velocity^2*length);  %vertical force (even numbered entries)
    n=n+1;
end
time=(starting_time:time_step:final_time)*timestep;
force_smooth(:,1)=smooth(force(:,1),9);
force_smooth(:,2)=smooth(force(:,2),9);
plot(time,force(:,1),'c-'); hold on; plot(time,force(:,2),'m-');
plot(time,force_smooth(:,1),'k-'); plot(time,force_smooth(:,2),'k-')
xlabel('Time (wingbeats)'); ylabel('Non-Dimensional Force'); legend('Horizontal','Vertical')
```

The public `Reading-Forces/dataread2.m`, shortened (the plotting of the half-stroke lines is left out). Read it on the next slide.

THE COURSE'S READER, READ

- one `F` file per loop, `textscan` with three header lines and `P` as a comment character;
- odd entries x, even entries y; summed; times $-2/(\rho U^2 L)$;
- `num2str(k)` writes `200000`, so it only finds files from step 10,000 up: below that the files have leading zeros (`F.01000`). The lab's 3D script said so in its header (slide 25);
- `timestep` and `time_step` are different things (the step size and the dump interval), a trap the lab's scripts carried for years;
- `smooth(..., 9)` is a 9-point moving average (Curve Fitting Toolbox);
- "Time (wingbeats)" because the run's unit of time was one wingbeat.

A READER THAT DOES NOT COUNT HEADER LINES: PYTHON

```
# examples/analysis/hier_data.py
def read_vec(path, ndim=2):
    """One X.NNNNN or F.NNNNN file as an (npts, ndim) array."""
    vals = []
    with open(path) as f:
        for line in f:
            s = line.strip()
            if not s or s[0].isalpha() or s.startswith("Process"):
                continue  # "Vec Object: ...", "  type: ...", "Process [k
  ]"
            try:
                vals.append(float(s))
            except ValueError:
                pass
    v = np.array(vals)
    if v.size % ndim:
        raise ValueError("%s holds %d numbers, not a multiple of %d" % (
  path, v.size, ndim))
    return v.reshape(-1, ndim)

def read_series(folder, kind="X", dt=None, ndim=2):
    """All X.* (or F.*) files of a folder, in time order: (times, [arrays
  ])."""
    files = sorted(glob.glob(os.path.join(folder, kind + ".*")), key=
  lambda p: int(p.rsplit(".", 1)[1]))
    steps = np.array([int(p.rsplit(".", 1)[1]) for p in files])
    t = steps * dt if dt is not None else steps
    return t, [read_vec(p, ndim) for p in files]
```

- A line is kept if it converts to a number; the header and the `Process` lines do not, whatever their count. The result is an array with one row per point and `ndim` columns, so `X[:, 0]` is every x and `F[:, 1].sum()` the summed y force.
- `read_series` sorts the files by their step number (not alphabetically, which would put `X.10000` before `X.00800` if the zeros were ever missing) and returns the times from `DT`; `read_input('input2d', 'DT')` fetches it.
- Run as a script, `python3 hier_data.py <run folder>` prints the frame count, the point count and the summed force at three times, which should match the log's `integral{F}`.

This is the reader behind every measured figure of chapters 7–10 (`tools/ch*_figures.py`).

THE SAME READER IN MATLAB AND OCTAVE

```
% examples/analysis/read_hier_data.m
function [t, data] = read_hier_data(folder, kind, dt, ndim)
%   [t, X] = read_hier_data('hier_data_IB2d', 'X', 1.0e-4);      % 2D
  positions
%   [t, F] = read_hier_data('hier_data_IB3d', 'F', 1.0e-4, 3);   % 3D
  forces
if nargin < 3, dt = []; end
if nargin < 4, ndim = 2; end
files = dir(fullfile(folder, [kind '.*']));
steps = zeros(numel(files), 1);
for k = 1:numel(files)
    steps(k) = str2double(files(k).name(numel(kind) + 2:end));
end
[steps, order] = sort(steps);
files = files(order);
if isempty(dt), t = steps; else, t = steps * dt; end
data = cell(numel(files), 1);
for k = 1:numel(files)
    lines = strsplit(fileread(fullfile(folder, files(k).name)), {'\n', '\r
  '});
    v = str2double(lines);              % NaN for "Vec Object: ...", "
  type: mpi", "Process [0]", ""
    v = v(~isnan(v));
    if mod(numel(v), ndim) ~= 0
        error('read_hier_data:count', '%s holds %d numbers, not a
  multiple of %d', files(k).name, numel(v), ndim);
    end
    data{k} = reshape(v, ndim, [])';     % one row per point: x y [z]
end
end
```

- `str2double` of a line with letters is `NaN`, so the header and `Process` lines drop out; no `HeaderLines`, no `commentstyle`, no toolbox.
- `data{k}` is an `npts`-by-`ndim` matrix at time `t(k)`: `sum(F{k})` is the summed force, `X{k}(1, :)` the first point.
- `read_input.m` fetches `DT`, `RHO`, `V` from the input file (a plain number only; `MU = 2.0*PI*...` is an expression and is not evaluated).
- Octave 8.4 runs it unchanged; `leaf_forces.m` (next slides) uses it and prints the same table as the Python version to the printed digits.

THE WORKED EXAMPLE: DRAG AND LIFT ON THE TETHERED LEAF (IB/03)

```
# examples/analysis/leaf_forces.py (the MATLAB version does the same)
dt = read_input(os.path.join(run, "input2d"), "DT")
rho = read_input(os.path.join(run, "input2d"), "RHO")
U = read_input(os.path.join(run, "input2d"), "V")      # the flow speed
  the boundary conditions tend to
L = 0.1                                                 # the leaf's
  length (generate_mesh2d.py)
t, F = read_series(os.path.join(run, "hier_data_IB2d"), "F", dt)
t, X = read_series(os.path.join(run, "hier_data_IB2d"), "X", dt)
F_sum = np.array([f.sum(axis=0) for f in F])            # what the
  examples print as integral{F}
CD = -F_sum[:, 0] / (0.5 * rho * U ** 2 * L)
CL = -F_sum[:, 1] / (0.5 * rho * U ** 2 * L)
top = np.array([x[0] for x in X])                        # point 0, the
  top end (y = +0.05 at t = 0)
bot = np.array([x[-1] for x in X])                       # the last point,
   the bottom end
```

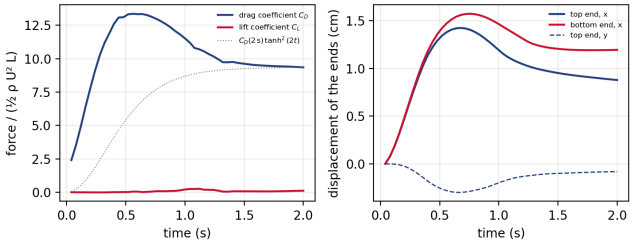
Six lines of reading, two of normalizing, two of picking points. IB/03: the leaf held at its middle in a stream that ramps up as $V\tanh(2t)$ to 0.1 m/s, $\rho = 1000$, $\mu = 4$, $\mathrm{Re} = 2.5$ (chapter 7).

What the two lines of output mean:

- C_D is positive: the fluid pushes the leaf downstream and the two tethers pull it back by the same amount;
- C_L is near zero and changes sign: the leaf is symmetric about the stream, and the small lift comes from the asymmetry of its bending;
- the ends' displacement shows the bending: the top and bottom ends move downstream by the same amount while the middle is held, and the beam's stiffness sets how far.

`python3 leaf_forces.py <run folder>` and, in MATLAB or Octave in the run folder, `leaf_forces`; both write `leaf_forces.png`.

IB/03: THE FORCE AGAINST TIME



Left: C_D and C_L against time; the dotted curve is $C_D(2\,\mathrm{s})\tanh^2(2t)$, what a force proportional to $U(t)^2$ would do. Right: the displacement of the leaf's two ends.

Measured (both scripts agree to the printed digits): C_D rises with the ramp to 13.1 at $t = 0.48$ s, where the stream has reached 75% of its speed and the leaf is still bending, and decays to 9.3 at 2 s (mean 9.8 over the second half); C_L stays within ± 0.2. The ends move 1.3 cm downstream at 0.48 s and settle at 0.9 cm. In the 2013 convention ($F/(\rho U^2 L)$, half of these) the force at 2 s is 4.7.

The 2013 lecture's figure for this leaf ("Dimensionless force vs. time, $\mathrm{Re} = 2.5$", over 20 s, from an impulsive start) showed the drag spiking to 7 in the first half second, ringing, and decaying to a plateau of 1.6 by $t = 8$ s, with the lift at zero. The ramped start of this series' IB/03 removes the spike and the ringing; the decay is the same, and the plateau is reached only after several seconds, which the homework runs.

IB/03 as shipped, IBAMR 0.19.0, 3 processes; `examples/analysis/leaf_forces.py` (the MATLAB version gives the same table).

POSITIONS: KINEMATICS FROM THE X FILES

Everything measured in chapters 7–10 came from the `X` files with the reader of slide 18:

- a point: the leaf's ends (chapter 7), the beam's tip (chapter 10): `X[k][i]` at time `t[k]`;
- an extent: the band's diameter (chapter 8a), the plates' gap (chapter 9a), the tube's smallest opening (chapter 9b: the minimum over i of $y_{\mathrm{top},i} - y_{\mathrm{bottom},i}$);
- a cloud: the pump's 121 markers, whose mean x measured the net transport (chapter 9b), the fruit-fly plate's centroid and angle;
- a shape against its target: the curling beam's distance from its arc, the bell's width (chapter 10);
- a velocity: differences of positions over the dump interval, $(\mathbf{X}_i(t_{k+1}) - \mathbf{X}_i(t_k))/(t_{k+1} - t_k)$: crude but often enough; IBAMR's own `U` data could be written the same way as `X` (`getLData("U", ...)`) if you need it per step.

The 2013 MATLAB lecture's `positionread.m` and `readpositions.m` did this for one time ("returns vectors holding the x and y coordinates of the boundary points at one instance in time"), and `pcolor_frames.m` overlaid the bell's points on the vorticity at ten times, "really specific to jellyfish swimming: the y-position of the top of the bell is read, and all y-values are shifted. The top of the bell is always at position $y = 0$." That shift, subtracting a point's position from the others', is how a swimmer's shape is compared from pulse to pulse.

SEVERAL STRUCTURES IN ONE FILE

```
# IB/07: two plates of 103 points, in the order of structure_names
t, F = read_series("hier_data_IB2d", "F", dt)
left, right = F[k][:103], F[k][103:]              # the index ranges
  of chapter 9a
print(left.sum(axis=0), right.sum(axis=0))        # equal and
  opposite pulls

# IB/11: the driving piece first, then tube (860), pericardium
  (1066), racetrack (3436), markers (121)
nd = 82                                           # actuator_top +
  actuator_bot in mode TARGET_VP
i_tube = nd
i_mark = nd + 860 + 1066 + 3436
tube = X[k][i_tube:i_tube + 860]
markers = X[k][i_mark:i_mark + 121]
work = sum((F[k][:nd] * (X[k + 1][:nd] - X[k][:nd])).sum() for k in
  range(len(t) - 1))   # the pump's W
```

The files hold every structure, one after another, in the order of `structure_names`; the point counts are the first lines of the `.vertex` files. So:

- the sum over *all* points mixes the forces on every structure: in IB/07 the two plates' pulls cancel, and the log's small `integral{F}` hides two forces of $63{,}000$ N (chapter 9a's README);
- sum over a structure's slice to get its force; sum over a sub-range for a part of it (the driven piece of the pump);
- the work a driven piece does, $W = \sum_{\text{steps}} \sum_i \mathbf{F}_i \cdot \Delta\mathbf{X}_i$, needs both files at consecutive dumps and the dump interval small enough that $\Delta\mathbf{X}$ follows the motion: chapter 9b's pump coefficient;
- if `structure_names` changes between runs (IB/11's five input files), the slices move: read the counts from the `.vertex` files rather than hard-coding them.

FILTERING

"Just like real data, immersed boundary points have noise due to vibrations. This is a smoothing function; control it with `windowSize`." (the lab's 2014 3D script)

```
% the course (2012): a 9-point moving average, Curve Fitting
  Toolbox
force_smooth(:,1)=smooth(force(:,1),9);
% the lab (2014): a 90-point moving average with filter(), no
  toolbox; it lags by 45 samples
windowSize = 90;
force(:,1)=filter(ones(1,windowSize)/windowSize, 1, force(:,1));
```

```
# Python: a centred moving average of w samples (NumPy only)
w = 9
kernel = np.ones(w) / w
CD_smooth = np.convolve(CD, kernel, mode="same")   # the ends are
   biased; drop w//2 samples there
```

Where the ripple comes from, and what to do:

- the IB force on a point is a spring or tether force, and the points jitter by a fraction of a grid cell as they are advected across cells; the delta function makes the spread force depend on where in a cell a point sits. The ripple has the period of a point crossing a cell, $\Delta x/U$, and grows with stiffness;
- it is noise in the sense that it is not in the continuum problem; it averages to nothing over a few samples, and a moving average of 5–20 samples removes it. Make the dump interval short enough that the ripple is sampled ("more frequent than visit outputs for filtering");
- `filter()` is one-sided and shifts the signal by half the window; `smooth()` and `np.convolve(..., 'same')` are centred;
- do not filter away the physics: a wingbeat sampled 20 times per period with a 9-point window loses its peaks. Plot raw and filtered together, as the course's figure did.

IN 3D: THREE NUMBERS PER POINT, AND THE LAB'S SCRIPT

```
% a 2014 lab script for hier_data_IB3d (the header notes, shortened):
% If the script fails to run, the following are likely causes of
  failure:
% 1. If n=1 and fileID=-1, then the script was unable to open the first
   file:
%   a. Check the path_name ... don't end your pathname with a '/' ...
%   b. Check the starting_time ... you MUST have an F.10000 file in
  your data folder.
%   c. Check that the starting_time is 10,000 or greater. The data
  files for
%      time steps less than 10,000 have a leading zero (i.e. F.01000),
  which
%      MATLAB does not automatically write out.
% 2. If only some data is written out: check that the time_step from
  input2d is correct.
% 3. If your data isn't what you expected: double check length,
  velocity, density.
while ~(fileID==-1)
    fileID = fopen([path_name{1} '/hier_data_IB3d/F.' num2str(k)]);
    if (fileID==-1), break, end            % the end of the folder
    C=textscan(fileID,'%f','HeaderLines',3,'delimiter',' ','
  commentstyle','P');
    A=C{1}; fclose(fileID);
    force(n,1)=-2*sum(A(1:3:end))/(density*velocity^2*length);   % x
    force(n,2)=-2*sum(A(2:3:end))/(density*velocity^2*length);   % y
    force(n,3)=-2*sum(A(3:3:end))/(density*velocity^2*length);   % z
    n=n+1; k=k+time_step;
end
```

- In 3D the folder is `hier_data_IB3d` and every point has x, y, z in turn: `A(1:3:end)`, `A(2:3:end)`, `A(3:3:end)`; the readers of this series take `ndim = 3`.
- The script's normalization divides by $\rho U^2 L$ with L a length: in 3D the coefficient wants an area S (chapter 8b, $F/(\rho U^2 L^2)$); the script's "force" is then $C \cdot L$, fine for comparing runs of one geometry, wrong for comparing with a published C_D.
- The while loop stops at the first missing file, so a run that is still writing is read as far as it got.
- Its debugging notes are the three mistakes everyone makes: the path, the first file's number, the leading zeros. The series' readers list the folder instead of counting.

THE 2013 MATLAB LECTURE: EULERIAN DATA TOO

From *Converting IBAMR output to MATLAB and analyzing it* (April 2013, and the 2020 Lecture 5):

- "Eric Tytell has Matlab code that reads data in the `visit_dumps`, interpolates it, and outputs Matlab structures. When you open the folder, you need to run `install_here.m`. The relevant Matlab files are in the CFD folder. I have added some additional files for reading and analyzing `hier_data_2d`."
- "It is necessary to not output Div U when running IBAMR due to the space in the name: `OUTPUT_DIV_U = FALSE`."
- "`[x,y,Vinterp,V] = importsamrai(pathname,'interpolaten',[GridSizex GridSizey])`: imports samrai data from files in the directory pathname. Files should be named `summary.samrai` and `processor_clusterXXXXX.samrai`. `V` is the original visit data, and `Vinterp` is the interpolated data. The output `V` is a structure containing one element for each grid region: `V.level_number`, `V.xlo`, `V.xup`, `V.rows`, `V.cols`, `V.U_0`, `V.U_1`, `V.P`, `V.F0`, `V.F1`, `V.Omega`."

THE 2013 MATLAB LECTURE: ITS FUNCTIONS

- `Forceread.m`: "takes the path name and the number of time steps and returns vectors holding the forces in the x and y directions at each of the boundary points for one instance in time." `positionread.m`: the same for the positions. (`readpositions.m` on a later slide is the same function under its other name.)
- `Force_vs_time.m`: "will read in the force data from `hier_data_IB2d` and will determine the average dimensionless drag force acting on the structure vs. time. The force on each boundary point is summed. The high Re approximation that $F = C\rho SU^2$ is used to rescale the force." (The leaf figure of slide 21's caption came from it.)
- `pcolor_frames.m`: "reads in positions from `hier_data` and vorticities from `viz_IB2d` samrai data and makes a 5×2 plot of vorticity at 10 times. This function is really specific to jellyfish swimming."
- 2020: `flappy_ends.m` ("imports SAMRAI data into Matlab, interpolates to a uniform grid, reads in the Lagrangian positions, and calculates the volumetric flow rate and maximum velocity at the end of the tube") and `vol_flow_rate.m`, with the run's grid size, domain bounds, dump interval and point counts set by hand at the top of the script (`GridSizex = 4096`, `time_step = 5000`, `timestep = 7.5e-6`, `pperitop = 5080`): the pattern of slide 17, with the same `timestep`/`time_step` pair.

The Lagrangian half of that lecture is this chapter; the Eulerian half (SAMRAI files into MATLAB, interpolation to a uniform grid, flow rates across a line) is chapter 33, where the `importsamrai` tools are re-examined against VisIt's own export, which has become the simpler route.

OTHER WAYS IBAMR GIVES YOU FORCES AND POSITIONS

- The Silo Lagrangian output (`LSiloDataWriter`, registered in every example's `main.cpp`): `viz_IB2d/lag_data.visit`, the structures as meshes for VisIt (chapter 26). Positions only, at the visualization interval; VisIt can export them, but the `X` files are the direct route.
- The upstream examples' own post-processing: `examples/IB/explicit/ex2` (a cylinder) computes its drag and lift inside the program each step and writes them to a text file; the IBFE examples `ex5`, `ex6`, `ex11` do the same with the finite-element force (Part IV). The pattern is a function called after `advanceHierarchy` that sums the forces it wants and appends a line: a lighter alternative to dumping every point when only the totals are needed.
- Callbacks: the integrator accepts functions to run before or after each step (chapter 9b used the pre-step hook for the pump); a post-step callback can sum forces or sample positions without touching the main loop.
- Restart files hold everything but are not for analysis.
- IBFE (Part IV) has its own force and position output through libMesh's ExodusII files and VisIt; chapter 20's examples show it.

ENERGIES AND WORK FROM $\mathbf{F}$ AND $\mathbf{X}$

With both files at every dump you can compute the quantities the chapters before asked for:

- work done by a driven structure (chapter 9b's pump coefficient): $W = \sum_{\text{dumps}} \sum_{i \in \text{driven}} \mathbf{F}_i \cdot \big(\mathbf{X}_i(t_{k+1}) - \mathbf{X}_i(t_k)\big)$; the dump interval must resolve the motion, and the sign tells whether the structure pushes the fluid or is pushed;
- stretching and bending energy (chapter 10's homework): $E_s = \sum \frac{1}{2} K_{\text{file}} (R - R_0)^2$ over the springs and $E_b = \sum \frac{1}{2} K_{\text{file}} |\mathbf{X}_{i-1} - 2\mathbf{X}_i + \mathbf{X}_{i+1} - \mathbf{C}_i|^2$ over the beams, from the positions and the stiffnesses in the structure files (the file stiffnesses already carry the Δs factors, so these are energies per unit depth);
- power into the fluid: $P = \sum_i \mathbf{F}_i \cdot \mathbf{U}_i$, with $\mathbf{U}_i$ from differences of positions or from `getLData("U", ...)`;
- a force balance check: the sum over a tethered structure equals minus the fluid force; over a free structure it is zero; over everything in a periodic box it is the net force on the fluid, which must be balanced by a mean pressure gradient (chapter 7's note on periodic domains).

> Chapter 8b's dimensionless outputs (F', C_{pump}, Strouhal numbers) are all one of these divided by ρ, U and L. Compute them from the files, not by eye from VisIt.

WHAT CAN GO WRONG

The mistake	What happens
the sign	a drag that is negative: the file holds the force on the fluid; the force on the body is minus the sum
the $\frac{1}{2}$	coefficients twice too large or small, and the 2013 “divide by 2” myth. Say which convention you use
three header lines assumed	a file from one process may have fewer: the first x is lost and every point’s x and y swap. Keep lines that are numbers
`num2str(k)` for the file name	no file below step 10,000 is found (`F.01000`); list the folder instead
files sorted as strings	`X.10000` before `X.00800` if leading zeros are missing; sort by the step number
all structures summed	the plates’ pulls cancel, the pump’s tube hides the actuator; slice by `structure_names`
`timestep` and `time_step`	the step size and the dump interval, confused in a dozen lab scripts; name them `dt` and `interval`
too coarse a dump	a wingbeat sampled 4 times per period looks like noise; no filter helps
too wide a filter	the peaks of a periodic force are flattened and shifted; plot raw and filtered
a loose Stokes tolerance	a force ripple that is not vibration; `-stokes_ksp_rtol 1e-12` (chapter 16) to test
3D force divided by a length	a coefficient times a length; use an area

HOMEWORK

From the 2012–2020 courses and this chapter (Appendix C collects the full set):

1. IB/03: run the course's 20 s (`END_TIME = 20.0`) and plot C_D to the plateau; compare with the 2013 figure's 1.6. Then halve μ and say what happens to the plateau and to the ripple.
2. IB/07: the force on each plate against time from the `F` files (chapter 9a's README table), with the gap from the `X` files on the same axes.
3. IB/11: the work done by the actuator over the second period and the pump coefficient (chapter 9b), for the target-point and the applied-force modes.
4. IB/10: the bell's vertical force (the sum over the bell, which should be zero) and the apex's velocity from differences of positions; is the jellyfish accelerating during contraction or expansion?
5. Write the energy of IB/09's beam from the `X` files and the stiffnesses, and plot it against time.

> What to hand in: the reader you used, the normalization (with ρ, U, L and the convention), the raw and the filtered curve on one plot, and one sentence on the sign of each force.

Extension: rerun IB/03 with `-stokes_ksp_rtol 1e-12` and overlay the two raw C_D curves.

RULES OF THUMB

- The file holds forces on points, on the fluid. Sum a structure's slice; negate for the force on the body.
- Keep lines that are numbers; sort by step number; the time is step $\times$ `DT`.
- $x, y[, z]$ alternate per point; in Python `[:, 0]`, `[:, 1]`; in MATLAB `(:, 1)`, `(:, 2)`.
- One normalization, stated: $C = F/(\frac{1}{2}\rho U^2 S)$ or $F' = F/(\rho U^2 L)$.
- Reproduce the log's `integral{F}` before trusting anything else.
- Dump often enough to see the ripple, filter it with a centred window, show both.
- Positions give kinematics; positions and forces together give work and energy.
- Structures sit in the file in input order; slice, never sum blindly.

SUMMARY

1. IBAMR's Lagrangian force is a force on each point, not a density, and the Δs in every stiffness is that conversion. The examples write it, with the positions, as PETSc ASCII vectors every `data_dump_interval` steps, in Lagrangian order, from their own `output_data()`.
2. The sum over a structure is its tether and applied force, minus the fluid's force on it; a free body sums to zero. The course's coefficient divides by $\frac{1}{2}\rho U^2 S$, chapter 8b's by $\rho U^2 L$; the "divide by 2" of 2013 was that $\frac{1}{2}$.
3. The readers of this series keep every numeric line, sort by step and return one row per point, in Python and in MATLAB; `leaf_forces` shows the whole analysis on IB/03 in a dozen lines, with the same numbers from both.
4. Positions give every kinematic quantity the earlier chapters measured; positions and forces give work and energy; several structures are slices of the same file.
5. Filter the cell-crossing ripple with a centred moving average and show the raw signal too; tighten the Stokes tolerance to tell ripple from solver noise.

Next, chapter 12: 3D simulations, where the files have three numbers per point and the structures are sheets and bristles.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The file format is shown from a current file and the writer is quoted; the “three header lines” of the course scripts is explained and replaced by readers that do not count them, in both languages, shipped in `examples/analysis` and tested.
- The 2013 “scaling error, divide by 2” is retired with its explanation; the sign and the $\frac{1}{2}$ are stated once.
- The fruit-fly example, no longer available, is described from its slides and figure (the figure itself, with a personal path in its title, is not reproduced); the method is demonstrated on IB/03 with a measured table and a comparison to the 2013 leaf figure.
- The lab’s 2014 3D script is quoted for its debugging notes and its 3D indexing, with its author’s personal path removed; its normalization by a length is flagged.
- New: what the sum of the forces means structure by structure, slicing several structures, work and energy from the two files, the origin of the ripple and how to filter it, the Stokes tolerance, and the other outputs IBAMR has.
- The Eulerian half of the 2013 MATLAB lecture is moved to chapter 33.

REFERENCES

- IBAMR: `examples/IB/explicit/ex1/example.cpp` (`output_data`, the origin of the examples' writer, with the HDF5 Eulerian output), `examples/IB/explicit/ex2` (drag and lift in the program); `include/ibtk/LDataManager.h` (`getLData`, `scatterPETScToLagrangian`), `LSiloDataWriter.h`. https://github.com/IBAMR/IBAMR; the FAQ, https://github.com/IBAMR/IBAMR/wiki/FAQ.
- PETSc: `VecView` and the ASCII viewer, https://petsc.org/release/manualpages/Vec/VecView/.
- This series: `examples/analysis/` (`hier_data.py`, `read_hier_data.m`, `read_input.m`, `leaf_forces.py`, `leaf_forces.m`); the `tools/ch*_figures.py` scripts of chapters 7–10; chapter 8b (dimensionless forces), chapter 9b (the pump coefficient), chapter 33 (Eulerian data into MATLAB).
- The public course folder `Reading-Forces`: `dataread2.m` (2012–2013) and the lab's 2014 3D script.
- Sources: *IBAMR Forces* (MATH 769, February 26, 2012, 8 slides) and its editions *IBAMR Tutorial: Output and Analyze Forces* (public folder 7, 2017) and *Lecture 15: Output and Analyze Forces* (2015); *Converting IBAMR output to Matlab and analyzing it* (MATH 769, April 23, 2013, 12 slides); *5: Reading forces and converting IBAMR output to Matlab and analyzing it* (2020, 24 slides).